

Travelling transporter problem

Introduction

For ride-hailing services and food delivery services like Uber and Takeaway.com, they have a set of trips from some starting location at a specific time (e.g. passenger pickup or restaurant), to some end point at a later time (e.g. passenger drop-off or customer location). A crucial problem they need to solve is to efficiently assign these trips to vehicles (or drivers). This has a direct and significant impact on their profit. Two potential goals would be:

- *Maximizing number of trips per vehicle*: If they are able to assign more trips to each vehicle, they need fewer vehicles to fulfil more (or all) trips. More trips per vehicle also means each vehicle (and also driver)'s time is utilized more efficiently.
- *Minimizing travel between trips*: We want to avoid travelling too far to get from the end location of one trip to the start location of the next trip. The more travel there is between trips, the less efficient vehicles and drivers are.

These goals are related: if there is less travel between trips, each vehicle is able to perform more trips.

In practice, it is likely better to try to calculate profit directly. Each trip likely has some given revenue, and there are known costs for vehicles or drivers and travel time, from which we're able to calculate profit. As a general solution, I'm not considering profit directly in this project.

Other problems could also be formulated through this trip-based lens. One example is delivering goods from some depots to other depots or to customers, although such problems may have more efficient solutions involving assigning trips to depots.

This is different from the well-known travelling salesperson problem, given that:

- We don't just need to visit specific locations, but travel from point A to point B.
- Trips have defined start times, rather than being able to take place at any time.
- There are multiple vehicles, not just a single one.

Problem description

For this project, I consider a more simplified version than what we'd likely see in the real world. For ride-hailing services for example, trips are commonly booked shortly before they take place, there usually is a variable amount of waiting time, and drivers start their day from a specific location. I instead suppose we have all trip information upfront, each trip has a fixed start and end, and vehicles can start from anywhere. This could still be useful for getting a rough estimate of the minimum number of vehicles needed and the maximum potential profit, to see how much potential room there is to increase profit.

I set the number of vehicles upfront.

Objective function: For this problem, I try to maximize the number of trips per vehicle.

Decision variables: For each trip and each vehicle, consider whether that trip is performed by that vehicle.

Constraints:

- Each trip should only be performed by one vehicle.
- For any two trips on a vehicle, the later trip should be reachable from the earlier trip. That is to say: the start time of the later trip should not be before the end of the previous trip plus the travel time to the later trip's start location.

Model formulation

I define variables as follows:

- t_{ij} : whether trip i is performed by vehicle j (0 or 1)
- $\overline{u_{ij}}$ is the set of trips incompatible with trip i , with u_{ijk} being whether trip k is performed by vehicle j
- s_i : start time of trip i
- e_i : end time of trip i
- d_{ij} : travel time between trip i and trip j
- n^t : the number of trips
- n^v : the number of vehicles
- n_{ij}^n : the number of trips incompatible with trip i on vehicle j

Note that the start, end and travel times are vehicle-independent.

Assume trips are sorted by end time. This leads to a more compact formulation, since for any trips i and j , if trip j has a greater index, it cannot be before trip i , since it will have an end time that's at least as large as the end time of trip i . This can be represented as follows:

$$e_i \leq e_j \quad \forall i, j \in n^t : i < j$$

So, to check that later trips are reachable from the current trip, we only need to consider greater trip indices.

Building on the above, we can say two trips t_{ij} and t_{kj} (with $k > i$) are incompatible if and only if the later trip's start isn't reachable from the earlier one's end. That is:

$$t_{kj} \in \overline{u_{ij}} \iff e_i + d_{ik} \leq s_k \quad \forall i, k \in n^t : i < k, \forall j = \{1, 2, \dots, n^v\}$$

To set constraints for all pairs of trips that are incompatible, we could use a simple $t_{kj} + t_{ij} \leq 1$, but this will cause an explosion of constraints. Below I use more compact constraints which combine a trip and all its incompatible trips into a single constraint.

$$\begin{aligned}
& \max \quad \sum_{i=0}^{n^t} \sum_{j=0}^{n^v} t_{ij} \\
& \text{s.t.} \quad \sum_{j=0}^{n^v} t_{ij} \leq 1 \quad \forall i = \{1, 2, \dots, n^t\} \\
& \quad \quad wt_{ij} + \sum_{j=0}^w u_{ijk} \leq w \mid w = n_{ij}^n \quad \forall i = \{1, 2, \dots, n^t\}, j = \{1, 2, \dots, n^v\} \\
& \quad \quad t_{ij} \in \{0, 1\} \quad \forall i = \{1, 2, \dots, n^t\}, j = \{1, 2, \dots, n^v\}
\end{aligned}$$

Method

The problem formulated above is a mixed-integer linear programming problem.

To generate an optimal solution, I used Python and the `milp` method from SciPy, which is an open-source library with various mathematical functions.

Larger-scale problems can't be solved using the above in a reasonable timeframe, given the complexity of the problem. To deal with this, I developed a greedy heuristic which goes through trips by sorted end time, and assigns each trip to the first vehicle where it fits (if any).

To get an upper bound of the objective value (which enables us to estimate the performance of the greedy heuristic), I generated the optimal solution for a single vehicle, and took that as the upper bound for the average number of trips per vehicle. A downside of this upper bound is that we expect the optimal objective value to decrease as the number of vehicles increases, but this upper bound stays constant.

Data collection and/or generation

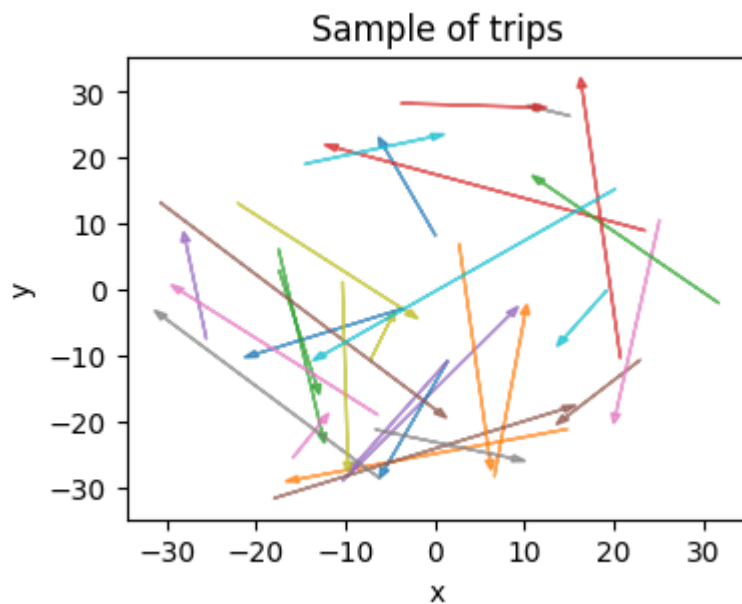
I used an existing dataset from [the OR dataset library](#), provided by Mohan Krishnamoorthy, Andreas Ernst and their collaborators. This provides collections of trips which is used directly for modelling.

The above dataset contains various problem instances of various sizes.

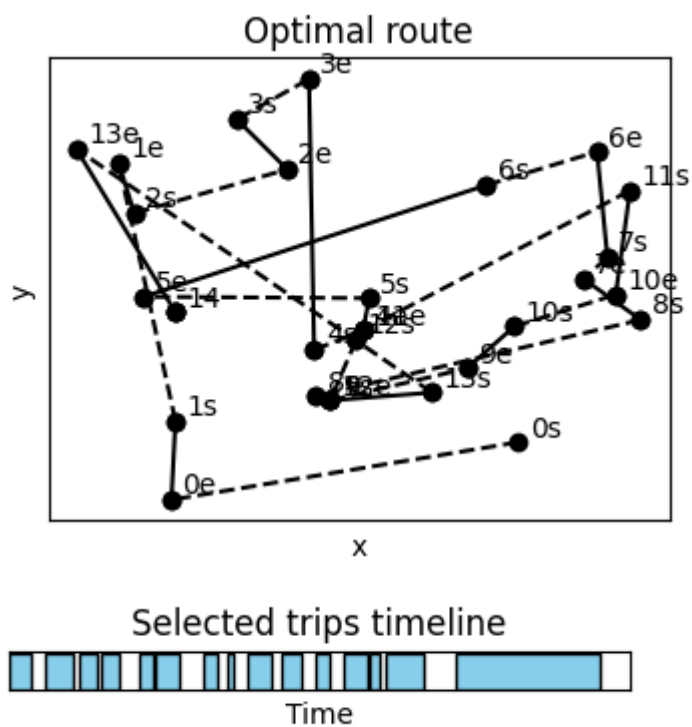
I used one of these instances for initial modelling (1500 trips), and additional ones for further evaluation.

Results

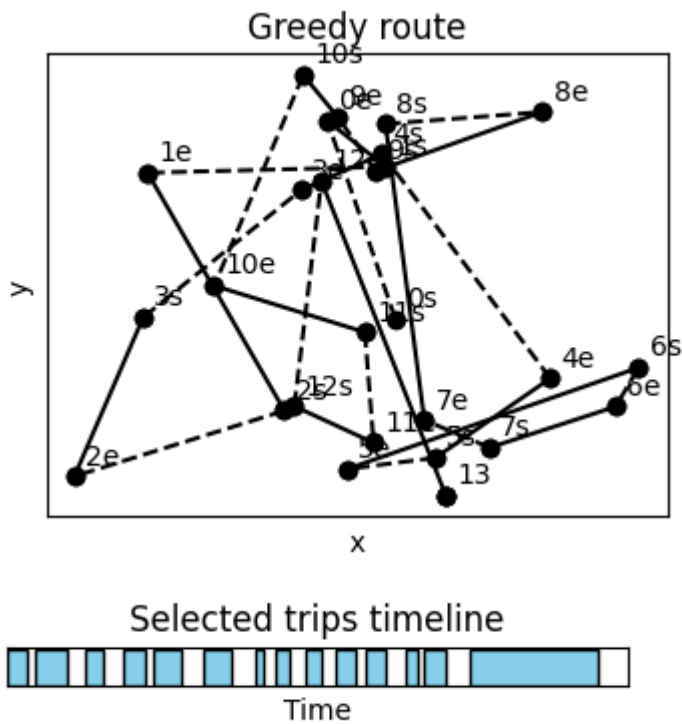
Here is a small sample of trips in our example instance.



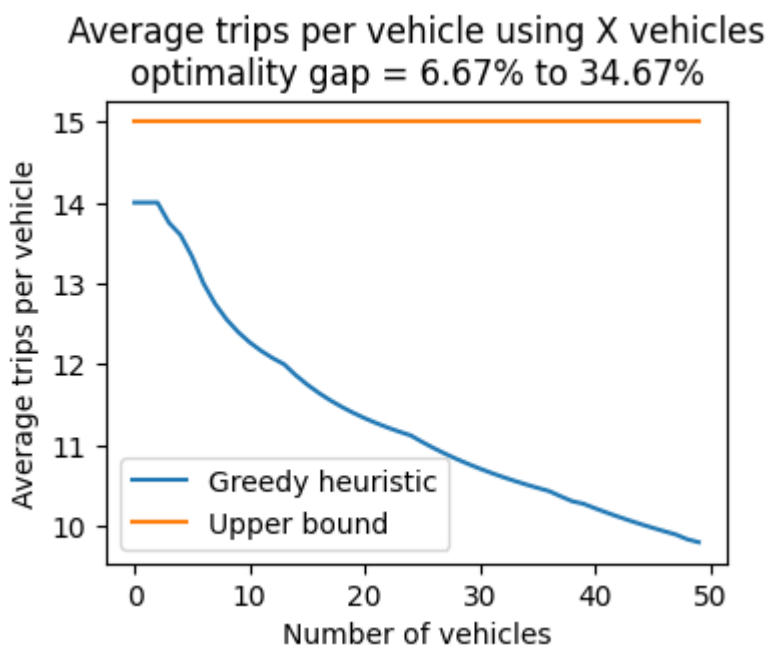
For 1 vehicle, the optimal route contains 15 trips. Here is a graphical plot of the route, along with a timeline of the trips on that route.



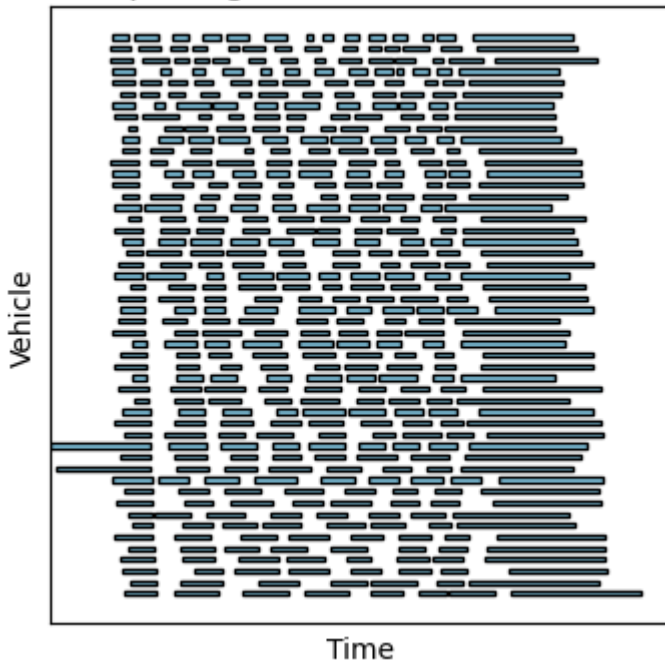
By comparison, our greedy algorithm contains 14 trips (optimality gap 6.67%). Here is a graphical plot and timeline of the route.



We can run our greedy algorithm for more vehicles, showing that the number of trips per vehicle decreases as the number of vehicles increases. This is expected, both because more vehicles with the same number of trips means the optimal solution is worse, and because our algorithm is suboptimal.

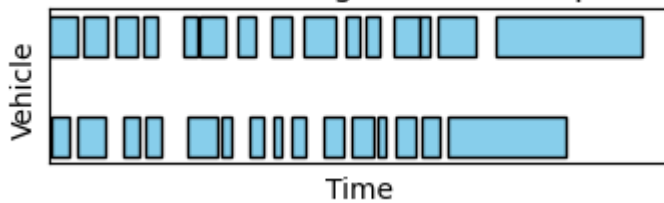


Trip assignment for each vehicle

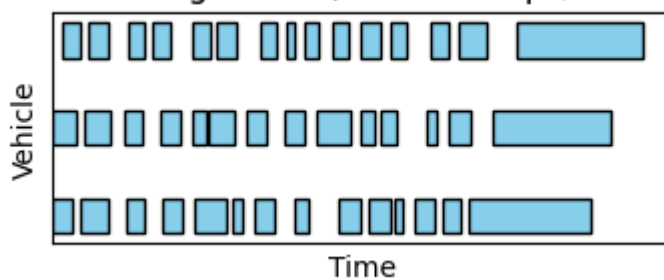


I also ran the integer programming solver for 2 and 3 vehicles. 2 vehicles, also resulted in 15 trips per vehicle. 3 vehicles resulted in 14.33 trips per vehicle. Although the latter timed out after running for 30 minutes. This is somewhat consistent with the greedy algorithm, which has 14 trips per vehicle for 1-3 vehicles.

Optimal 2-vehicle assignment (15 trips/vehicle)



Optimal 3-vehicle assignment (14.3333 trips/vehicle) (timed out)



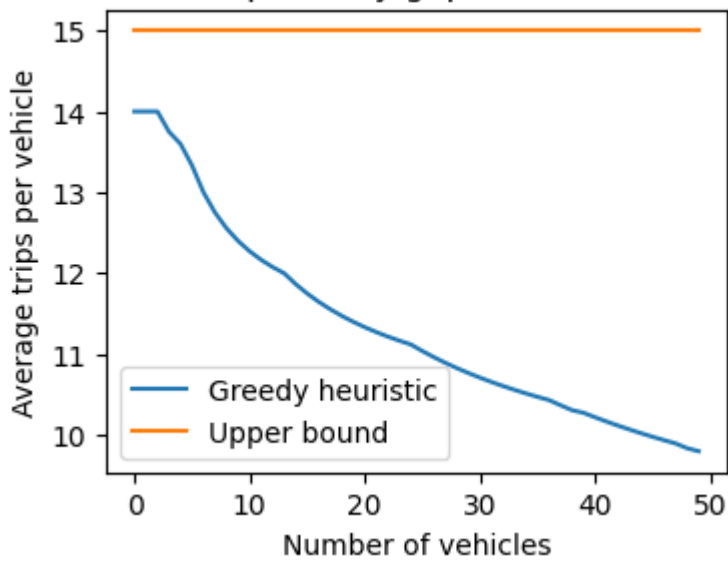
Other instances

I also ran the greedy and optimal 1-vehicle algorithms on other instances from the dataset, finding similar results.

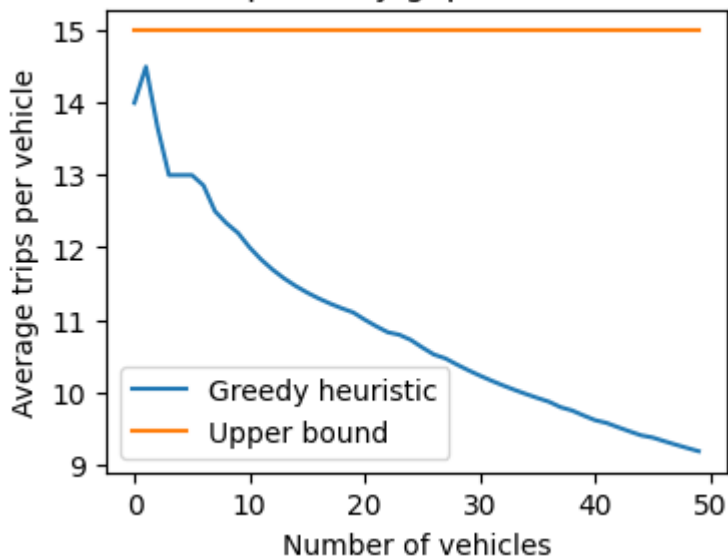
Each instance has 1500 trips, and for each subsequent instance, those trips are taken from a shorter period of time. This helps to show how the algorithm deals with more and less dense data.

The optimality gap ranges from 6.67% to 16.67% for 1 vehicle, up to 28.4% to 38.8% for 50 vehicles, with a relatively constant decline in performance as the number of vehicles increases.

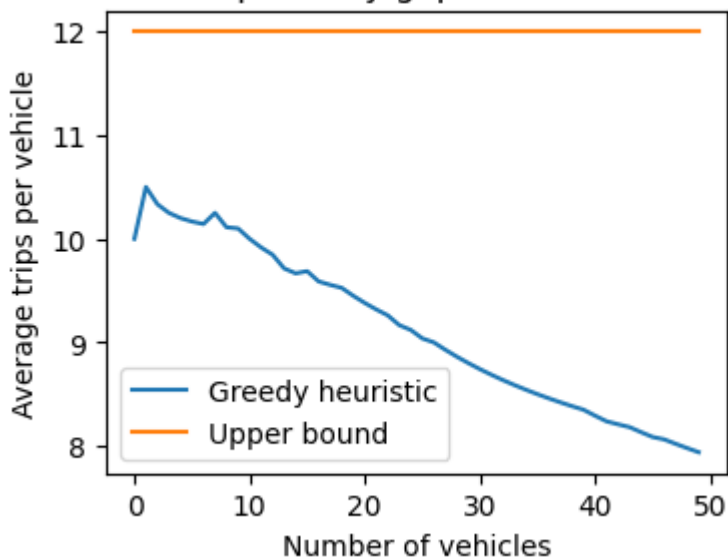
Instance 0, optimality gap = 6.67% to 34.67%

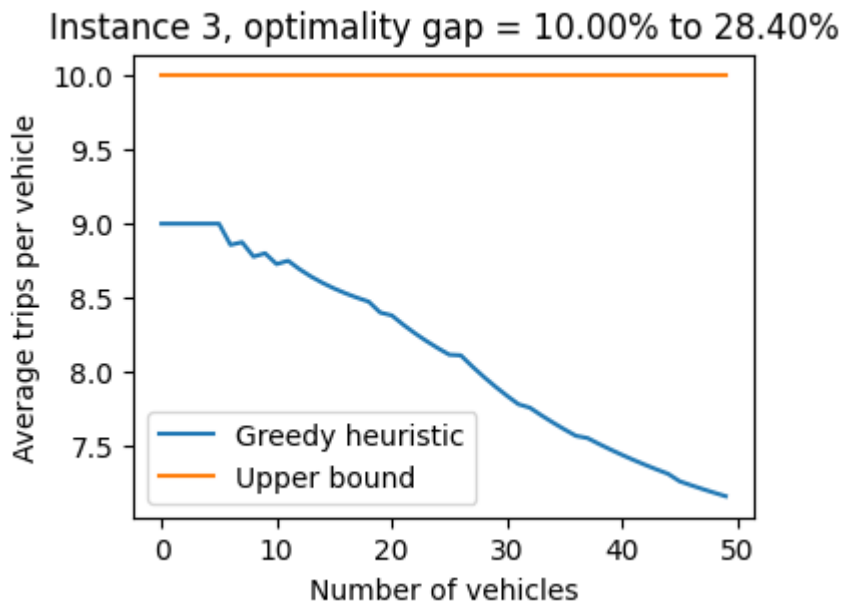


Instance 1, optimality gap = 6.67% to 38.80%



Instance 2, optimality gap = 16.67% to 33.83%





Conclusion

The results show that the greedy heuristic produces a reasonable solution. This could be used to get a rough estimate of the optimal solution for real-world applications.

Further work could involve processing trips as they come in, varying weighting time rather than fixing trip time, and possibly having some predefined driver start locations. This could better deal with the real-world constraints of use cases like ride-hailing.

One could also try to improve the upper bound to get a better estimate of how well the greedy heuristic performs as the number of vehicles increases.